

DPC-Sharding: Adaptive and Byzantine-Resilient Sharding for Dynamic Networks

Zhujun Zhang , Wei Fan , and Zhiqian Liu , *Senior Member, IEEE*

Abstract—Blockchain sharding faces inherent challenges in dynamic partially-connected IoT and edge computing networks, arising from static CAP tradeoff strategies, rigid Byzantine fault tolerance (BFT) quorum thresholds, and insufficient synergistic coordination between network dynamics and consensus execution. Existing adaptive sharding schemes adopt crude binary mode switching or heuristic threshold adjustment, which not only degrades consensus throughput and causes sharp performance degradation, but also introduces potential security risks and weak resilience under node churn and network partitioning. This paper proposes DPC-Sharding, a closed-loop adaptive sharding framework that leverages formal Multi-Threshold BFT (MT-BFT) safety invariants and features a dedicated clamped threshold control mechanism for dynamic networks. The framework integrates latency-sensitive state machine (LSSM) for quantitative network volatility perception, clamped sigmoid-based multi-threshold adaptive consensus, and a three-phase execution stack equipped with volatility-aware topology-pruned BBA (VTP-BBA) and atomic state recovery mechanism. All dynamic consensus quorum thresholds are strictly clamped within a unified provable security interval, ensuring security consistency between protocol practical execution and formal safety verification. Extensive experiments compared with OmniLedger and DynaShard demonstrate that DPC-Sharding achieves comparable performance with state-of-the-art synchronous sharding protocols under stable networks, and delivers prominent robustness against high node churn and intermittent network partitioning. Furthermore, the framework guarantees cross-shard transaction atomicity throughout continuous network dynamics without global synchronization. This work provides a viable solution for edge and IoT blockchain sharding systems to address dynamic network challenges while complying with formal Byzantine security requirements.

Index Terms—Blockchain sharding, byzantine fault tolerance, dynamic networks, cross-shard atomicity, adaptive consensus.

I. INTRODUCTION

BLOCKCHAIN ensures global consistency through decentralized consensus without trusted intermediaries [1].

Received 11 November 2025; revised 30 May 2026; accepted 7 July 2026. Date of publication 14 July 2026; date of current version 1 September 2026. This work was supported by the National Key Research and Development Program of China under Grant 2021YFB2700603. (*Corresponding author: Wei Fan.*)

Zhujun Zhang and Wei Fan are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100089, China (e-mail: zhangzhujun@iie.ac.cn; fanwei@iie.ac.cn).

Zhiqian Liu is with the College of Cyber Security, Jinan University, Guangzhou 510632, China, also with the Guangdong Key Laboratory of Data Security and Privacy Preserving, Jinan University, Guangzhou 510632, China, also with the Guangdong Hong Kong Joint Laboratory for Data Security and Privacy Protection, Jinan University, Guangzhou 510632, China, and also with the Guangxi Key Laboratory of Trusted Software, Guilin University of Electronic Technology, Guilin 541004, China (e-mail: zqliu@vip.qq.com).

Digital Object Identifier 10.1109/TDSC.2026.3713485

Conventional blockchain designs suffer from limited scalability, while the ideal assumptions of network stability and full connectivity hardly hold in practical dynamic distributed systems [2], [3]. Typical application scenarios including decentralized IoT, edge computing, and vehicular networks experience frequent node churn and intermittent links, which partition the network into temporally isolated shards and degrade consensus liveness [4], [5], [6].

To tackle the above issues, sharding has become a mainstream solution. However, most existing designs still rely on static network assumptions and cannot well accommodate inherent network fragmentation [7]. This leads to a common dilemma: high-performance synchronous consensus degrades under network fluctuations, while resilient asynchronous protocols introduce excessive latency, making it difficult to satisfy latency-sensitive and robustness-demanding applications such as mobile payments and edge data sharing.

Challenges in Existing Approaches: Classic principles such as CAP and PACELC reveal inherent consistency–availability trade-offs [8]. Even so, current sharded consensus still suffers three key drawbacks in dynamic partially connected networks:

Fragile Static Sharding: Most static sharding schemes adopt fixed partitioning and epoch-based reconfiguration, lacking the ability to adapt to node mobility and unexpected offline events. This raises intra-shard latency and reconfiguration overhead, and weakens cross-shard coordination.

Rigid Quorum-Based BFT: Fixed quorum thresholds like $\lceil 2n/3 \rceil$ are sensitive to honest node fluctuation under node churn, even if the Byzantine ratio remains within $f < n/3$. Existing partially synchronous dynamic BFT mainly focuses on membership reconfiguration [9], while most adaptive BFT designs adopt binary mode switching [10], [11], [12], [13], easily causing throughput fluctuation during state transitions.

Synchronous-Asynchronous Dichotomy: Synchronous consensus achieves low latency under stable connectivity but faces liveness risks under network partitions. Asynchronous protocols enhance partition resilience yet maintain high latency in normal scenarios, limiting overall throughput and complicating cross-shard processing [14], [15], [16], [17].

Research Objective: We propose DPC-Sharding (Dynamic, Partially-connected, and CAP-aware Sharding), a dynamically adaptive sharding framework that considers the efficiency of synchronous-inspired consensus and the resilience of asynchronous-inspired quorum design under standard Byzantine fault tolerance. The core objective is to build a closed-loop design where network state sensing drives formally constrained,

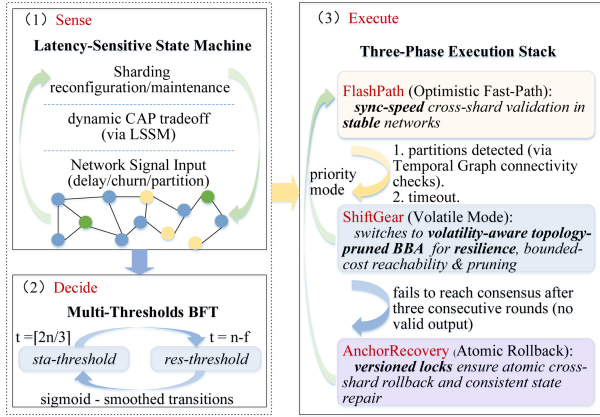


Fig. 1. The overview of DPC-Sharding. The multi-threshold BFT mechanism is formally bounded and clamped within the safety constraints derived from Multi-Threshold BFT (MT-BFT) [18], ensuring uniform security even under dynamic adaptation.

Note: *sta-threshold* denotes the quorum bound for stable-mode $t_{sta} = \lceil 2n/3 \rceil$, while *res-threshold* denotes the quorum bound for resilient/volatile mode $t_{res} = n - f$.

clamped BFT threshold calibration, which in turn triggers safe execution-mode shifts—enabling seamless adaptation while preserving safety and efficiency.

As shown in Fig. 1, DPC-Sharding integrates three synergistic innovations:

- A Latency-Sensitive State Machine (LSSM) that quantifies network volatility and binds CAP behavior to temporal stability, eliminating heuristic timeouts.
- A Clamped Multi-Threshold BFT mechanism that derives its safe bounds from MT-BFT [18] and enforces threshold clamping to ensure smooth, sigmoid-based transitions always comply with safety invariants.
- A Three-Phase Execution Stack that coordinates FlashPath (high-throughput consensus), ShiftGear (topology-aware BBA), and AnchorRecovery (atomic state recovery).

These modules form a closed-loop control system: LSSM network signals drive safe threshold adaptation, which governs execution modes. This ensures adaptation is uniform, secure, and formally consistent at all times.

The Specific Problem Addressed: This work addresses robust, efficient sharded consensus in dynamic, partially connected networks under standard BFT assumptions ($f < n/3$). We solve three key challenges:

- Adapt to continuous network volatility without performance cliffs.
- Maintain valid quorum formation and liveness despite node churn.
- Ensure atomic cross-shard execution and consistent recovery.

Specific Contributions: Our work advances the state of the art with four key contributions:

- *Temporally bounded CAP adaptation via LSSM:* We present a network state machine that ties consensus behavior to temporal stability, reducing misclassification and removing heuristic timeouts.

- *Smooth clamped multi-threshold BFT under formal safety bounds:* We enable sigmoid-based continuous quorum calibration strictly confined within the formal MT-BFT safe interval. This clamped adaptation guarantees provable security throughout dynamic tuning and avoids abrupt performance cliffs.
- *MT-BFT Clamped Threshold-Based Topology-Aware Consensus:* We design a volatility-adaptive binary Byzantine agreement protocol that reduces redundant communication via topology pruning. It supports synchronized threshold adjustment and versioned voting with all operational overhead bounded.
- *Atomic cross-shard recovery via versioned locks:* We implement consistent state rollback and fault repair mechanisms to sustain cross-shard transaction atomicity under continuous network volatility.

DPC-Sharding does not rely on ad hoc threshold choices. All adaptive thresholds are formally clamped within MT-BFT safety bounds, ensuring alignment between protocol design, safety arguments, and complexity analysis under a uniform fault tolerance model. We validate DPC-Sharding via theoretical analysis and extensive experiments.

While our framework establishes a continuous threshold space within the formal safety interval, we adopt several discrete representative quorum levels in practical implementation to reduce runtime tuning overhead. All discrete levels are strictly clamped within the same MT-BFT safety bounds and follow the sigmoid interpolation principle, ensuring consistency between theoretical design and practical realization.

The rest of this paper is organized as follows. Section II reviews related work. Section III presents the system model and preliminaries. Section IV introduces DPC-Sharding. Section V provides theoretical analysis. Section VI evaluates performance. Section VII concludes the paper.

II. RELATED WORK

We categorize existing works into four groups: static sharding, asynchronous partition-resilient consensus, adaptive BFT/dynamic sharding, and network topology perception.

Static Sharding Protocols: OmniLedger [19] and RapidChain [15] adopt fixed BFT thresholds and static structures, performing well in stable networks while having limited adaptability to frequent node churn. Related studies pursue single-dimensional optimization: Estuary [20] reduces cross-shard traffic via state splitting; FortunChain [21] adopts static sharding and fixed consensus mechanisms; BrokerChain [22] mitigates hot shards with dynamic partitioning; Reputation-based Sharding [23] optimizes shard composition relying on node reputation evaluation; ContribChain [24] optimizes load balancing via clustering. None of them adopt temporal topology. DPC-Sharding introduces temporal topology to enable dynamic shard adjustment with low overhead under network fluctuations.

Asynchronous Partition-Resilient Consensus: This class of protocols prioritizes liveness under network partitions. Komorebi [25] implements DAG-based BFT with high overhead; Fin [26] develops efficient asynchronous common subset

consensus. DPC-Sharding balances latency, overhead and resilience via multi-threshold BFT and latency-sensitive state machines.

Adaptive and Dynamic Sharding Mechanisms: Existing designs support basic adaptation but seldom consider network topology evolution. DynaShard [11] reorganizes shards dynamically yet brings non-negligible reconfiguration overhead. SmartChain [27] uses Markov decision processes with limited capability for abrupt network changes. Sabine [28] optimizes validators via machine learning without topology awareness. Mercury [29] employs binary operation modes and cannot interpolate intermediate quorum thresholds. DPC-Sharding builds a closed-loop framework with temporal clustering, volatility quantification and consensus feedback to achieve fine-grained adaptation.

Temporal Graph Theory and Sharding Applications: Temporal graph theory [30], [31] is widely used for dynamic connectivity modeling in non-blockchain scenarios [32], [33], while its integration with sharding remains limited. Representative graph-based schemes lack temporal modeling: BSTVG [34] is designed for device clustering; GDSS [35] uses static partitioning; AdaBFT [36] adjusts thresholds empirically. These works separate topology perception from consensus execution. DPC-Sharding unifies temporal clustering, sigmoid-based volatility evaluation and failure-aware consensus in one architecture.

In summary, existing solutions handle network dynamics, consensus and topology independently, including dedicated designs for mobile IoT [37] and vehicular networks [38]. Few integrate dynamic sensing, adaptive consensus and topology awareness into one framework. DPC-Sharding adopts a unified closed-loop design to support provable security and stable performance under persistent network volatility.

III. SYSTEM MODEL AND PRELIMINARIES

This section presents the foundational models and assumptions of our dynamic sharding framework, aligned with edge/IoT deployment characteristics [39].

A. Network Model

We model the dynamic edge/IoT blockchain as a partially connected temporal graph [40]. It captures time-varying latency, intermittent links and continuous node churn, forming transient partitions without global network paralysis.

Definition 2.1. Temporal Connectivity: Two nodes u and v are temporally connected at time t if a valid communication path exists with latency below a time-varying threshold $\tau(t)$. The threshold differentiates intra-shard and cross-shard communication [41].

B. Consensus and Blockchain Requirements

Safety and Liveness: Honest nodes reach consistent agreement on transaction ordering and state transition, maintaining liveness even under partial network partitioning.

Cross-Shard Atomicity: Cross-shard transactions follow causal execution to avoid inconsistent state forks [42].

State Recovery: Nodes achieve global state synchronization via AnchorRecovery, with cryptographic proofs ensuring auditability and tamper resistance [43], [44], [45], [46].

C. Temporal Graph and Valid Shard Definition

We employ temporal graph metrics to quantify link stability and support valid shard partitioning. Network latency and link stability are measured via sliding window statistics and exponential smoothing.

Definition 2.2 (Valid Shard): A shard S is a valid consensus subgroup if: (1) it maintains stable temporal connectivity; (2) malicious nodes are strictly less than $1/3$; (3) intra-shard latency remains steady for continuous consensus. Shard reconfiguration is triggered once the above constraints are violated.

D. MT-BFT Safety Envelope and Quorum Bound

We extend the MT-BFT framework to establish a unified safety envelope for adaptive quorum calibration [18]. Two invariant bounds are defined as follows:

- **Stable Safety Bound:** $t_{\text{sta}} = \lceil 2n/3 \rceil$ for consistency-oriented stable consensus.
- **Resilient Liveness Bound:** $t_{\text{res}} = n - f$ to preserve liveness under network partitions.

All intermediate adaptive thresholds across network transitional phases are generated via clamped sigmoid interpolation and strictly confined within the secure interval $[t_{\text{sta}}, t_{\text{res}}]$. All adaptive adjustments follow the constraints of boundary clamping, safety invariant preservation, monotonic variation, and extreme state convergence, ensuring provable security for all runtime threshold configurations under standard Byzantine quorum systems [47].

E. Network Volatility Quantification

We adopt a continuous quantitative model to characterize network fluctuation. The shard-level volatility score ω_s quantifies relative latency jitter, and a normalized stability confidence $\alpha \in [0, 1]$ is derived from multi-window consensus as the global adaptive control signal [48].

F. Overhead and Complexity Model

Topology maintenance incurs auxiliary background overhead including reachability detection and subgraph construction. Since shard topology evolves much slower than consensus iteration, such cost can be fully amortized, keeping the core workflow at $O(n)$ communication complexity [49].

G. Threat Model for Dynamic Sharding

We adopt the standard Byzantine fault model, where each shard satisfies $n \geq 3f + 1$ with the fraction of malicious nodes strictly bounded by $f < n/3$ [39], [50]. Adversaries are capable of launching network eavesdropping, packet tampering, and intentional network partitioning attacks.

DPC-Sharding is designed and verified under this assumption with three defense mechanisms:

- *Intra-shard defense*: Periodic leader rotation and joint pre-verification mitigate malicious domination.
- *Cross-shard defense*: Versioned locks avoid partial commit anomalies.
- *Dynamic transition defense*: Sigmoid-based tuning and outlier filtering suppress system oscillation.

IV. DPC-SHARDING MECHANISM

DPC-Sharding integrates the core design philosophy of Dynamic Partition-tolerant Consensus into the sharding architecture, and proposes a dynamic sharding framework that adapts to network volatility via three interconnected DPC-driven mechanisms: temporally-bounded CAP tradeoffs for shard-level partition tolerance, dynamic BFT threshold calibration for hierarchical shard consensus, and a phased execution stack for full-lifecycle shard operation. All mechanisms take individual shards as the basic scheduling unit, enabling adaptive regulation of partition-tolerant capability in both intra-shard and cross-shard scenarios of the sharding system.

A. Temporally-Bounded CAP Tradeoffs Via Latency-Sensitive State Machine

This module acts as the state perception layer of DPC-Sharding and provides network state awareness for adaptive partition tolerance. It takes individual shards as the granularity of volatility sensing and mode switching, tightly binding DPC's temporally-bounded partition tolerance to the temporal characteristics of shard-level network stability, thus enabling continuous adjustment of CAP tradeoff strategies for each shard in response to dynamic network conditions.

Compared with traditional static sharding systems (e.g., OmniLedger) and parallel BFT schemes that rely on fixed switching thresholds (e.g., PARBFT), the proposed Latency-Sensitive State Machine (LSSM) improves adaptability to network fluctuations through quantitative volatility measurement and statistically consistent state judgment. LSSM establishes a continuous stability evaluation mechanism to avoid sharp performance drops and frequent oscillations caused by binary mode switching.

LSSM achieves latency-sensitive adaptation through three core components: volatility quantification, Byzantine-resilient distributed parameter consensus, and calibrated temporal boundaries. It maintains consistency-oriented configurations in stable network conditions, enhances availability in volatile conditions, and ensures secure and seamless transitions between modes under unified Byzantine fault tolerance constraints.

Volatility Sensing: To enable reliable and oscillation-free mode transitions, LSSM first quantifies instantaneous network jitter via latency fluctuation measurement, then derives a global stability confidence score through multi-window statistical consensus.

Definition 4.1. Volatility Score: For a shard s , let $(l_1, l_2, \dots, l_m)$ be latency measurements (in milliseconds) taken at 1-second intervals over a 5-second window. The volatility

score is defined as:

$$\omega_s = \frac{1}{\bar{l}_s} \sqrt{\frac{1}{m-1} \sum_{i=1}^m (l_i - \bar{l}_s)^2} \quad (1)$$

where $\bar{l}_s$ denotes the mean latency. This metric reflects short-term relative network jitter and ensures cross-shard comparability despite different baseline latency levels. A shard is classified as stable if $\omega_s \leq \omega_{th}$, where ω_{th} is calibrated based on empirical stable network traces; otherwise, the shard is regarded as volatile. Such classification directly guides CAP tradeoff strategies: stable networks prioritize consistency, while volatile networks tend to enhance availability.

Based on consecutive multi-window observations of ω_s , LSSM further computes a normalized stability confidence score $\alpha \in [0, 1]$, which serves as the unified global control signal for the entire DPC mechanism. This score is derived from statistical consensus results rather than heuristic configuration:

$$\alpha = \frac{W_{\text{stable}}}{W_{\text{total}}}$$

where W_{stable} denotes the number of consecutive sliding windows that achieve valid honest consensus confirmed by at least $n - f$ nodes, and W_{total} denotes the total number of monitored sliding windows. A higher α indicates sustained and credible network stability, while a lower α reflects persistent volatility and potential partitioning risks. This stability score α is directly adopted by the dynamic threshold mechanism to support sigmoid-based smooth mode scheduling, guaranteeing behavioral consistency between state perception and consensus decision-making.

Byzantine-Resilient Mode Transition: To prevent malicious nodes from forging network states and triggering invalid mode switching, LSSM adopts a validation rule fully consistent with Byzantine safety constraints.

Theorem 4.1: For a shard with $n \geq 3f + 1$ nodes, a valid mode transition is triggered only if more than $n - f$ nodes consistently agree on the volatility status over a sustained period. This ensures that an adversary controlling fewer than f malicious nodes cannot falsify network states or forge transition decisions, and all mode transitions faithfully reflect real network conditions.

CAP Tradeoff Rules: Mode transitions between stable and volatile states are governed by consistent agreement on sustained volatility scores, with hysteresis constraints introduced to avoid periodic oscillation.

- *Stable $\rightarrow$ Volatile*: Triggered if more than $n - f$ nodes report $\omega_s > \omega_{th}$ for k_1 consecutive windows of duration w . The transition prioritizes availability to accommodate prolonged network partitioning.
- *Volatile $\rightarrow$ Stable*: Triggered if more than $n - f$ nodes report $\omega_s \leq \omega_{th}$ for k_2 consecutive windows. The transition reverts to consistency-prioritized protocols to maximize consensus throughput.

Distributed Parameter Consensus Protocol: To reach secure agreement on volatility score ω_s in Byzantine environments

while balancing overhead and decentralization, this paper designs two optional parameter consensus schemes that both comply with the Byzantine constraints in Theorem 4.1.

BFT Aggregation Scheme: This conventional scheme adopts core-node aggregation and lightweight BFT consensus to realize global consistency of ω_s , which is efficient in network environments with stable core nodes but suffers from single-point failure and broadcast overhead.

Reverse Consensus Scheme: The decentralized parameter consensus architecture is proposed to eliminate single-point failure risks of core nodes and reduce full-network broadcasting overhead, adapting to resource-limited scenarios such as edge computing. It adopts the latest block timestamp and transaction hash as the unified spatiotemporal anchor across the entire network. All nodes generate synchronous latency sampling rules based on identical hash seeds and reuse existing heartbeat channels for latency data collection. Each node independently verifies whether its local ω_s falls within the globally consistent interval via an $O(1)$ complexity consistency check function. If verification fails, neighbor sampling weights are adaptively adjusted until global convergence, eliminating explicit parameter reporting and aggregated consensus. A CRS cycle binding mechanism is deployed for dual identity and data verification: malicious nodes will cause their computed ω_s to deviate from the global normal interval due to forged latency data, and their cumulative verification failure counts are recorded on the blockchain. Once the count exceeds the predefined threshold, malicious nodes are automatically eliminated from the shard. This mechanism achieves Byzantine security and implicit self-exposure of malicious nodes without additional consensus rounds.

Overhead and Complexity Analysis: The local verification, adaptive weight adjustment, and periodic malicious node elimination operations introduce only constant per-round computational overhead and do not require full-network broadcasting or multi-round aggregated consensus. The network reachability detection and subgraph matching required for spatiotemporal anchoring are executed as background lightweight maintenance tasks, whose amortized overhead is negligible and does not affect the overall $O(n)$ linear communication complexity of the protocol. Therefore, the scheme retains high scalability while realizing decentralized and Byzantine-resilient parameter consensus.

State Machine Logic for Temporal Boundaries: The LSSM implements temporally-bounded CAP tradeoffs based on a finite state machine with three operational states: Stable (consistency-prioritized), Volatile (availability-prioritized), and Transitioning (specially designed to suppress oscillation during mode shifts).

Calibrating Temporal Boundaries: The partition tolerance time window τ determines the maximum duration of consistency prioritization under network partitioning, and is dynamically calibrated to match real-time CAP tradeoff demands:

Stable mode: $\tau = \tau_{\min}$, configured as the 99th percentile of round-trip latency under stable network conditions. A smaller time window enforces strict consistency by limiting the tolerated partition duration before switching to liveness prioritization.

Volatile mode: $\tau = \tau_{\text{base}} + \alpha \times k$, where $\tau_{\text{base}} \geq 2\tau_{\min}$ serves as the baseline tolerance to accommodate transient network

Algorithm 1: LSSM operation for temporal boundaries.

```

1:  $current\_state \leftarrow \text{Stable}$ 
2:  $\tau \leftarrow \tau_{\min}$ 
3:  $valid\_reports \leftarrow \{r \in peer\_reports \mid \text{verify}(r) = T\}$ 
4: if  $|valid\_reports| < n - f$  then
5:   return  $current\_state$ 
6: end if
7:  $volatile\_cnt \leftarrow |\{r \in valid\_reports \mid r_\omega > \omega_{th}\}|$ 
8: if Curr State = Stable then
9:   if  $volatile\_cnt \geq n - f \wedge \text{consec\_windows} = k_1$  then
10:     $trigger\_trans(target\_state)$ 
11:     $ramp\_start \leftarrow \text{now}()$   $\triangleright$  Start sigmoid
12:     $\tau \leftarrow \tau_{\text{base}} + \alpha \times k$ 
13:    return Transitioning
14:   end if
15: else if Curr State = Volatile then
16:   if  $volatile\_cnt < n - f \wedge \text{consec\_windows} = k_2$  then
17:     $trigger\_trans(target\_state)$ 
18:     $ramp\_start \leftarrow \text{now}()$ 
19:     $\tau \leftarrow \tau_{\min}$ 
20:    return Transitioning
21:   end if
22: else if Curr State = Transitioning then
23:   if  $\text{trans\_complete}(ramp\_start, r = 5s) = T$  then
24:    return  $target\_state$ 
25:   end if
26: end if
27: return  $current\_state$ 

```

instability; k denotes the number of disconnected components in the temporal network graph; α acts as a scaling factor calibrated according to the typical network recovery time of each fragmented component.

Transitioning mode: The temporal boundary τ inherits the value of the previous state and is linearly adjusted step by step toward the target state. This design avoids consensus execution disorder caused by abrupt boundary switching, balances atomicity guarantee and synchronization efficiency during mode transition, and realizes seamless state switching.

Overall, this design enables τ to adaptively scale with network instability: it increases with network fragmentation to prioritize availability, and decreases under stable conditions to enforce stricter consistency constraints.

B. Clamped Dynamic Thresholds Under MT-BFT Safety Bounds

Built upon the network volatility perception and stability confidence score α yielded by the preceding LSSM module, this layer acts as the decision control layer for the DPC sharding architecture. It adaptively adjusts consensus quorum thresholds for intra-shard transaction ordering, cross-shard atomic validation, and global blockchain checkpointing, taking α as

the core scheduling signal to hierarchically regulate partition tolerance. Under dynamic shard-level network fluctuations, this framework realizes mode-adaptive security-efficiency tradeoffs while strictly abiding by formal MT-BFT safety invariants.

Our design extends the static formal safety bounds of MT-BFT [18] to a runtime adaptive threshold scheduling paradigm. Critically, all adaptive quorum selections are strictly constrained within the unified MT-BFT safety envelope. This framework defines two canonical boundary thresholds as secure anchors: the stable safety bound $t_{\text{sta}} = \lceil 2n/3 \rceil$ for well-connected stable network scenarios, and the resilient liveness bound $t_{\text{res}} = n - f$ for volatile and partition-prone network environments. To achieve smooth and theoretically rigorous network adaptation, this work adopts clamped safe interpolation to generate continuous valid quorum thresholds between the two secure bounds. Different from empirical heuristic smoothing, all intermediate quorum values are fully formalized and constrained by four unified theoretical rules to guarantee provable security throughout adaptation.

Definition 4.2. MT-BFT Clamped Safe Interpolation: Given the stability confidence score $\alpha \in [0, 1]$, a continuously adaptive quorum threshold $t(\alpha)$ is MT-BFT secure if and only if it satisfies all of the following four constraints:

- **Boundary Clamping Constraint:** The threshold is strictly bounded within the MT-BFT safe region: $t_{\text{sta}} \leq t(\alpha) \leq t_{\text{res}}$ for all $\alpha \in [0, 1]$.
- **Strict Invariant Compliance:** All interpolated thresholds strictly adhere to the Byzantine quorum intersection property proven in Lemma 5.1, ensuring persistent safety during threshold adaptation.
- **Monotonicity Constraint:** The threshold varies monotonically with network stability: $t(\alpha)$ is non-decreasing with α , such that more stable networks adopt more consistency-oriented quorums and volatile networks gracefully shift toward liveness-oriented resilient quorums.
- **Extreme Convergence Constraint:** The interpolation strictly converges to canonical secure bounds at extreme network states: $t(\alpha) \rightarrow \lceil 2n/3 \rceil$ when $\alpha \rightarrow 1$ for fully stable environments, and $t(\alpha) \rightarrow n - f$ when $\alpha \rightarrow 0$ for highly volatile partition-prone environments.

In this work, the smooth threshold adaptation is implemented via sigmoid-based clamped interpolation:

$$t(\alpha) = \text{clamp} \left(\sigma(\alpha) \cdot \left\lceil \frac{2n}{3} \right\rceil + (1 - \sigma(\alpha)) \cdot (n - f), \left\lceil \frac{2n}{3} \right\rceil, n - f \right) \quad (2)$$

where $\sigma(\cdot)$ Denotes the standard sigmoid function. This formulation strictly satisfies the four constraints above, enabling macroscopic smooth adaptation without introducing unsafe quorum values or empirical parameter tuning. All layer-level threshold adjustments throughout the DPC-Sharding framework follow this unified interpolation rule and strictly abide by the two core MT-BFT safety invariants. In all operating modes, the interpolated quorum thresholds are rigorously clamped within the formal safe region and never fall below the resilient Byzantine-resilient bound $t_{\text{res}} = n - f$, guaranteeing persistent

Byzantine resilience and theoretical consistency throughout the entire adaptive process.

In this work, the quorum threshold is adjusted in a continuous and smooth manner. Compared with discrete threshold switching, continuous adjustment eliminates abrupt parameter mutations and reduces runtime jitter. Different from linear interpolation, nonlinear curves allow us to customize the adjustment speed under diverse network states. We select the Sigmoid function for weight calculation due to its three advantages. First, its output is bounded within $(0, 1)$, which helps constrain the interpolated threshold to a predefined interval. Second, the inherent S-shaped characteristic makes the threshold vary slowly at two ends and accelerate in the middle region, fitting the gradual evolution of network status. Third, the function is globally smooth, which benefits theoretical analysis. All thresholds generated by this strategy fall into $[t_{\text{sta}}, t_{\text{res}}]$. The standard Sigmoid function $\sigma(x)$ satisfies $x \in \mathbb{R}$ and $\sigma(x) \in (0, 1)$, and it is strictly monotonically increasing over the entire real domain. This inherent monotonic property guarantees that the interpolated threshold $t(\alpha)$ varies monotonically with the stability confidence α , which is fully consistent with the monotonicity constraint.

Hierarchical BFT with Dynamic Threshold Calibration: The framework is divided into three functional layers with differentiated responsibilities and hierarchical quorum adaptation. Guided by the stability confidence α , each layer performs continuous threshold tuning following the unified MT-BFT clamped safe interpolation rule, enabling smooth security-efficiency tradeoffs instead of brittle binary mode switching. All interpolated quorum values are bounded within the MT-BFT safety envelope and comply with the four interpolation constraints, ensuring theoretical validity throughout dynamic adaptation.

Layer 1 (Intra-Shard Fast Consensus): This layer handles high-throughput transaction ordering within individual shards and takes t_{sta} and t_{res} as two canonical anchor bounds. Following the defined sigmoid clamped interpolation, the quorum threshold adjusts continuously between the two bounds governed by α . Under stable well-connected conditions, the threshold converges to $\lceil 2n/3 \rceil$ for low-latency consensus; under volatile partition-prone conditions, it asymptotically approaches the resilient bound $n - f$. All intermediate interpolated values strictly satisfy the MT-BFT invariant constraints.

Layer 2 (Cross-Shard Validation): This layer guarantees transaction atomicity across multiple shards. Its threshold t_2 is dynamically determined following the weakest-link principle: $t_2 = \max(t_1^{(i)})$ over all involved shards i , where each $t_1^{(i)}$ is adaptively interpolated within its own MT-BFT safe region. If any shard remains in a volatile state, the elevated interpolated t_1 of that shard automatically raises t_2 to enforce rigorous cross-shard agreement. All dynamically derived cross-shard thresholds remain strictly confined within the unified MT-BFT safety envelope.

Layer 3 (Global Checkpointing): This layer periodically finalizes cross-shard block states to limit global state divergence, adopting a fixed checkpoint threshold $t_3 = 2f + 1$ that satisfies MT-BFT asynchronous safety requirements. Given the basic fault model $n \geq 3f + 1$, we can derive $2f + 1 \leq n - f$. Thus t_3 lies within the MT-BFT safe interval and complies with the

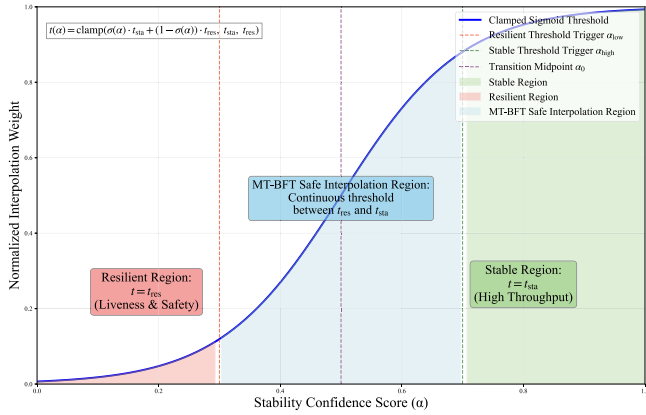


Fig. 2. Clamped sigmoid-based smooth adaptation of quorum threshold in Layer 1, governed by stability confidence score α . The quorum threshold is interpolated between two canonical secure bounds ($t_{sta} = \lceil 2n/3 \rceil$, $t_{res} = n - f$) under MT-BFT safety constraints. For illustration, we set $\alpha_{low} = 0.3$, $\alpha_0 = 0.5$, $\alpha_{high} = 0.7$ as exemplary configuration. This curve illustrates clamped quorum threshold adaptation within the MT-BFT formal safety envelope. All interpolated intermediate values are strictly bounded between $n - f$ and $\lceil 2n/3 \rceil$, fully comply with MT-BFT invariant constraints, and explicitly avoid unproven majority-level thresholds or empirical heuristic tuning.

safety bound, meanwhile preserving the quorum intersection property. Instead of adjusting quorum size, this layer adaptively tunes checkpoint frequency: a higher frequency under severe volatility accelerates partition recovery, while a lower frequency in stable mode reduces persistent synchronization overhead.

Dynamic Calibration Mechanism: Coordinated by the LSSM network state and the derived stability confidence α , multi-layer threshold adaptation is consistently orchestrated under the unified MT-BFT clamped interpolation rule. Layer 1 enables smooth quorum adjustment within the safe bound interval, Layer 2 maintains conservative cross-shard thresholds following the weakest-link principle, and Layer 3 adaptively tunes checkpointing frequency without altering its fixed secure quorum. All quorum parameters at runtime are continuously interpolated and strictly bounded within the predefined MT-BFT safety region.

Smooth Transition of Layer 1 Threshold: As illustrated in Fig. 2, the sigmoid function governs the clamped interpolation of Layer 1 quorum threshold driven by stability confidence α . The curve characterizes continuous threshold adaptation within the MT-BFT safety envelope, rather than merely adjusting the macroscopic ratio between two discrete fixed thresholds. All intermediate threshold values are formally verified and comply with the unified interpolation constraints.

Cross-Layer Coordination: Layer 1 achieves flexible performance-safety tradeoffs via probabilistic mode switching, while Layer 2 preserves global consistency through conservative threshold elevation. Layer-wise threshold differences reflect hierarchical security consolidation rather than inconsistent specifications: Layer 1 employs two native MT-BFT quorum bounds; Layer 2 inherits the most conservative valid quorum among participating shards following the weakest-link rule; Layer 3 adheres to the universal resilient-mode safety bound. All layer-level thresholds constitute valid subsets of the unified MT-BFT

Algorithm 2: Clamped Dynamic Threshold Orchestration.

```

1:  $\alpha \leftarrow$  LSSM stability confidence score
2:  $t_{sta} \leftarrow \lceil 2n/3 \rceil$ ,  $t_{res} \leftarrow n - f$   $\triangleright$  MT-BFT formal safety bounds
3: if  $\alpha > \alpha_{high}$  then  $\triangleright$  Stable: steady-state quorum
4:    $t_1^{target} \leftarrow t_{sta}$ 
5: else if  $\alpha < \alpha_{low}$  then  $\triangleright$  Volatile: resilient quorum
6:    $t_1^{target} \leftarrow t_{res}$ 
7: else  $\triangleright$  Transition regime: clamped sigmoid interpolation
8:    $t_1^{target} \leftarrow \text{clamp}(\sigma(\alpha) \cdot t_{sta} + (1 - \sigma(\alpha)) \cdot t_{res}, t_{sta}, t_{res})$ 
9: end if
10:  $t_2^{target} \leftarrow \max(t_1^{(i)})$   $\triangleright$  Cross-shard quorum
11:  $t_3^{target} \leftarrow 2f + 1$   $\triangleright$  Global checkpoint threshold
12:  $\mathcal{T}_{target} \leftarrow \{t_1^{target}, t_2^{target}, t_3^{target}\}$ 
13: return  $\mathcal{T}_{target}$ 

```

safety envelope, aligning formal security proofs, protocol logic, and runtime adaptive behavior.

In summary, the α -driven calibration ensures adaptation responds only to persistent network trends rather than transient noise. The system adopts asymmetric conservative convergence: it rapidly strengthens Byzantine resilience upon emerging instability and relaxes safety guarantees only after sustained network stabilization. By adopting formally defined clamped safe interpolation within the MT-BFT safety envelope, the framework achieves smooth theoretical-driven adaptation without sacrificing Byzantine security, fully unifying protocol design, formal invariants, and practical runtime performance.

C. Hierarchical Consensus With Three-Phase Execution Stack

This module serves as the execution implementation layer of DPC for sharding architecture, coordinating LSSM volatility signals and multi-level BFT dynamic thresholds to design a three-phase adaptive execution stack for the sharding system. It maps the DPC's partition-tolerant strategy requirements (high throughput/liveness priority/atomic recovery) to three dedicated execution phases for shard states (stable/volatile/healing), realizing the fine-grained adaptation of DPC's partition-tolerant capability in the full lifecycle of sharding operation.

Dynamic phase switching (FlashPath/ShiftGear/Anchor Recovery) is coordinated with LSSM signals and multi-level BFT thresholds, enabling adaptive CAP tradeoffs. These phases are not rigid sequential but adaptive strategies activated by network conditions, enabling the system to balance efficiency in stable regimes and resilience under partitions while preserving end-to-end safety.

To address the key limitation of existing sharding protocols that rely on static or periodic reconfiguration and lack fine-grained adaptability to gradual node dynamics and workload variation, this subsection designs Volatility-Aware Topology-Pruned BBA (VTP-BBA) within the ShiftGear phase. By reusing version numbers and coordinating states with FlashPath and

AnchorRecovery, we extend VTP-BBA's capabilities to a three-phase execution stack, thereby achieving fine-grained adaptation to network fluctuations throughout the entire lifecycle and fulfilling the adaptive CAP trade-off objectives outlined above.

FlashPath: FlashPath is the DPC high-throughput execution phase for stable shards, activating when the LSSM classifies the network as stable ($\omega_s \leq \omega_{th}$), and leveraging stable network characteristics to maximize throughput while optimizing cross-shard verification efficiency. Its design capitalizes on predictable low-latency environments, mirroring the performance of optimized classical synchronous BFT protocols while retaining sharding's parallel transaction processing advantages, achieved through mechanisms tailored to minimize cross-shard overhead and maximize consensus efficiency under stable connectivity.

Cross-shard transactions are validated using compact state root Merkle proofs, reducing inter-shard validation communication complexity from $O(n)$ to $O(1)$ and eliminating redundant data transmission—a critical bottleneck in traditional sharded systems. Specifically, each shard maintains a root hash of its current state in a Merkle tree, and cross-shard transaction validation relies on a Merkle branch proof that links the transaction's state change to this root hash. Receiving shards verify the proof against their locally cached copy of the sender's state root, avoiding the need to query multiple nodes for confirmation. This design ensures a single proof (of size proportional to the tree depth, typically 20–30 bytes) suffices for validation, regardless of shard size.

Layer 1 BFT uses the synchronous threshold $t_1 = \lceil 2n/3 \rceil$ to order transactions in batches via a pipelined propose-prepare-commit workflow, eliminating idle intervals between consensus phases, a key driver of throughput. The pipeline executes in overlapping stages: while the current batch undergoes the prepare phase, the next batch is already in the propose phase, with pre-validated transactions cached to reduce latency. This overlap compresses the time between batch commitments to within the LSSM's minimum latency bound τ_{min} , ensuring throughput scales linearly with batch size. Additionally, the threshold t_1 guarantees safety under synchronous conditions, as it exceeds the maximum number of Byzantine nodes ($f < n/3$) that could disrupt consensus.

Global checkpoints (Layer 3) are triggered at intervals calibrated to network stability, signed by $t_3 = 2f + 1$ nodes to bound state history size, with checkpoint frequency minimized in stable mode to reduce overhead.

By leveraging pipelined BFT and Merkle proof verification in stable networks, this mechanism maximizes throughput and minimizes cross-shard overhead. In summary, the core design of FlashPath enables it to achieve near-theoretically optimal performance in synchronous environments. The formal guarantees on throughput lower bound and synchronous safety will be rigorously proven and quantitatively analyzed in Theorem 5.2 of the theoretical analysis chapter.

ShiftGear: ShiftGear is the liveness-prioritized execution phase of DPC-Sharding tailored for volatile shard scenarios. It activates once the LSSM module captures severe network

partitioning, rising network volatility ($\omega_s > \omega_{th}$), or degraded BFT consensus performance, prioritizing consensus liveness over throughput to avoid suspension of partitioned shards. Traditional BBA protocols suffer from fixed decision thresholds that cannot adapt to dynamic network fluctuations, topology-agnostic voting logic that introduces invalid node votes, and stale voting conflicts that lead to slow or unbounded convergence. To mitigate these issues, we propose the VTP-BBA protocol, integrating three collaborative DPC-oriented mechanisms: volatility-adaptive threshold calibration for network dynamics, topology pruning for invalid node elimination, and versioned voting for fast conflict resolution under broken shard connectivity. ShiftGear adopts lightweight 1-bit BBA, compressing complex transaction verification into a binary commit/abort decision and retaining only the core consensus signal, which greatly simplifies intra-shard decision logic. As shown in Fig. 3, these mechanisms support fine-grained adaptive consensus under time-varying shard network conditions.

Different from conventional BBA with fixed thresholds, VTP-BBA implements a mode-separated dual-threshold adaptive mechanism to balance liveness in volatile scenarios and strict safety under the unified MT-BFT security envelope $[t_{sta}, t_{res}]$. All final consensus commits comply with the standard Byzantine fault tolerance assumption $n \geq 3f + 1$, eliminating potential threshold inconsistency and security degradation.

- 1) *Volatility-Aware Node Pruning Phase:* This phase eliminates unreachable and timeout nodes via global topology awareness. All nodes conduct reachability detection under the LSSM dynamic delay threshold τ and broadcast local reachable sets R_i . The intersection of these sets forms the global feasible subgraph G^* , which filters out isolated and invalid nodes and maintains Byzantine fault tolerance as a Minimum Connected Dominating Set (MCDS). Guided by the stability confidence α , VTP-BBA adaptively adjusts its operational rules: under mild network fluctuations, it follows standard supermajority within the safe interval $[t_{sta}, t_{res}]$; under severe partitioning, $\lceil |G^*|/2 \rceil$ is adopted only as a lightweight preliminary criterion to accelerate node eligibility screening within G^* . Notably, this threshold serves merely as a pre-screening gate rather than a formal consensus quorum, and final Byzantine agreement strictly adheres to the $n - f$ bound of the MT-BFT safety interval. Only nodes inside G^* participate in subsequent consensus, and the overhead of topology maintenance is amortized to preserve the $O(n)$ core consensus complexity.
- 2) *Versioned Binary Voting Phase:* A leader is elected from the valid node set G^* to initiate binary proposal broadcasting. Ordinary nodes verify proposal validity and respond with Echo messages. After collecting enough valid Echo messages that meet the current mode-dependent threshold, each node broadcasts a Ready message. Once voting conflicts caused by stale messages are detected, timestamp-based version vectors are adopted for version-aware revoting to resolve conflicts rapidly. The final consensus decision v_{final} is confirmed only when

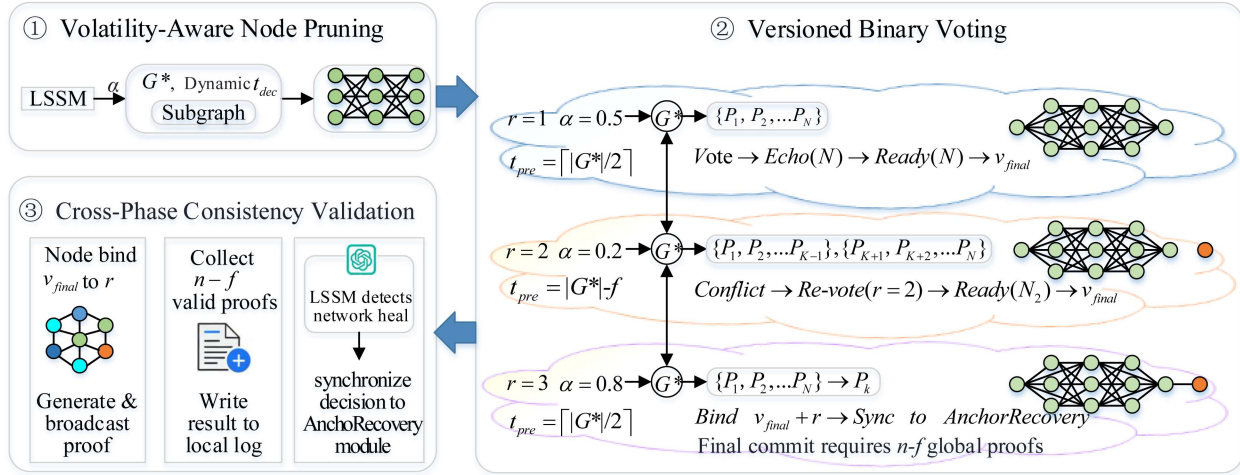


Fig. 3. VTP-BBA execution flow.

sufficient consistent Ready messages with the same version are collected. If consensus convergence fails, the iteration round number increments to trigger re-election and topology re-pruning, and the decision threshold is dynamically updated according to the latest α . This phase leverages monotonic version vectors to guarantee bounded convergence, with rigorous convergence bound analyzed in Theorem 5.4. This work defines the iteration round r as one-dimensional version vector and prove that the strict monotonic growth of r is the necessary and sufficient condition for shard consensus convergence. The detailed proof is given in Corollary 5.1, which addresses the unbounded iteration drawback of conventional BBA schemes.

- 3) *Cross-Phase Consistency Validation Phase:* Nodes bind the confirmed consensus result v_{final} with the iteration version number r to generate verifiable consistency proofs and broadcast them across the shard. A transaction result is persistently logged locally only after collecting more than $n - f$ valid global consistency proofs, which strictly anchors all consensus behaviors to the standard MT-BFT security invariant. Once the LSSM module detects network healing and topology recovery, the stabilized consensus state is immediately synchronized to the AnchorRecovery module, providing reliable version basis for atomic partition rollback and state recovery. This phase realizes closed-loop coordination between VTP-BBA consensus execution and AnchorRecovery fault recovery, ensuring end-to-end global consistency of the three-layer DPC sharding framework.

VTP-BBA incorporates the LSSM-generated stability confidence α into BBA threshold calibration to implement state-dependent adaptive threshold switching and avoid unconstrained threshold relaxation. The protocol constrains all stable-state consensus operations within the provably secure MT-BFT envelope $[t_{sta}, t_{res}]$. In transient network partitioning scenarios, $\lceil |G^*|/2 \rceil$ is only employed for lightweight prior node screening to improve liveness, whereas the formal consensus validation strictly follows the unified $n - f$ supermajority constraint to

maintain Byzantine security invariants. In addition, VTP-BBA supports topology-aware consensus through network reachability detection and valid subgraph aggregation, mitigating livelock problems caused by invalid and stale votes. The versioned iterative voting mechanism alleviates version conflicts and improves consensus convergence efficiency. Different from conventional BBA protocols that only support single-round consensus execution, the cross-module coordination between consensus and recovery maintains long-term state consistency and improves the fault tolerance capability in dynamic sharding networks.

AnchorRecovery: AnchorRecovery is the DPC atomic recovery execution phase for healing shards, serving as the complementary execution phase to ShiftGear, facilitating atomic rollback during partition healing. It activates when the LSSM healing signals (i.e., network partitions are resolved) and directly receives consensus results from the VTP-BBA protocol in ShiftGear. It addresses state divergence through versioned locks and eliminates the livelock problem in asynchronous fallbacks by ensuring cross-shard recovery proceeds atomically, thus forming a complete consensus-recovery closed loop.

We introduced the versioned locks to enforce atomicity and prevent stale state propagation. As a decentralized state coordination primitive, it is formally defined as $L = (tx_id, v, \sigma_v)$, where tx_id is a unique identifier for the cross-shard transaction, v is a monotonically increasing version number (initialized at 1 for new transactions, incremented with each shard hop), and σ_v is a multi-signature from more than t_2 nodes (Layer 2 threshold) to validate version authenticity.

By associating each transaction with a unique version v , nodes reject operations referencing outdated versions (e.g., a transaction with $v = 3$ is invalidated if a newer version $v = 5$ exists for the same state key). Cross-shard transactions can only commit if all involved shards agree on the latest version v , ensuring partial commits are impossible.

Versioned locks evolve through three stages.

- 1) *Initialization:* The initiating shard generates $L_0 = (tx_id, v = 1, \sigma_0)$ and broadcasts it to relevant shards.

- 2) *Propagation*: Each subsequent shard verifies $\sigma_0, \sigma_1, \dots, \sigma_{v-1}$, increments v by 1, and appends its signature to form σ_v .
- 3) *Finalization*: On commit, the maximum v across shards is embedded in the checkpoint; on rollback, all shards revert to the state referenced by the latest universally verified L_v .

To enable efficient cross-shard state synchronization during recovery, versioned state differences minimize data transmission: nodes exchange only changes incurred since the last checkpoint. These differences are authenticated via signatures from a number of nodes no fewer than the Layer 2 threshold t_2 , while cryptographic proofs of causality (e.g., vector clocks) prevent propagation of stale state versions.

For consistent merging of these differences, the causal merge protocol enforces dependency-based ordering of transactions— T_2 may depend on T_1 only if T_1 is finalized across all relevant shards—with timestamp priority resolving unavoidable conflicts to ensure deterministic convergence. Versioned locks enable quick identification and discarding of outdated differences (those with v less than the current checkpoint's baseline).

Finally, checkpoint finalization anchors the merged state as a new system-wide baseline, validated via signatures from a number of nodes no fewer than the Layer 3 threshold t_3 to guarantee irreversibility. The latest versioned locks are inscribed in this checkpoint, providing a trusted reference for subsequent transactions and preventing state divergence.

This three-phase design forms a closed loop with the LSSM and multi-level BFT: volatility signals drive phase activation, BFT thresholds enforce mode-specific safety, and execution outcomes feed back to refine network awareness. This synergy enables robust performance across dynamic network conditions.

AnchorRecovery mechanism utilizes version locks and a causal merge protocol to achieve atomic state synchronization and rollback after network partition recovery, thus resolving state divergence and eliminating livelock risks. This mechanism ensures the atomicity of cross-shard transactions and is crucial for the system's safe convergence from an anomalous state to a consistent one. Proofs of atomic commit property and partition healing convergence are presented in Theorems 5.5 and 5.7 respectively. The cross-phase global consistency property is demonstrated in Theorem 5.8.

D. Comparison With Baseline Designs

Temporal CAP Trade-off vs. Static Mode Adaptation: Existing sharding consensus schemes adopt static configurations or discrete binary switching strategies that cannot dynamically adjust CAP preferences according to temporal network volatility. Such rigid adaptation easily causes cross-shard congestion, unstable synchronization, and frequent state oscillation under node churn. In comparison, our LSSM enables quantitative volatility estimation and hysteresis-constrained temporal CAP adjustment, supporting fine-grained adaptation across stable, transitional, and volatile network states while maintaining consistent system robustness.

Algorithm 3: Three-Phase Execution Stack Transition.

```

1:  $\alpha \leftarrow$  LSSM stability confidence score
2:  $t_{\text{sta}} \leftarrow \lceil 2n/3 \rceil$ ,  $t_{\text{res}} \leftarrow n - f$ 
3: if  $\alpha > \alpha_{\text{high}}$  then ▷ High stability: FlashPath
   high-throughput phase
4:   return {phase: FlashPath,  $t_1 \leftarrow$ 
      $t_{\text{sta}}$ , checkpoint_freq  $\leftarrow$  stable_freq( $s$ )}
5: else if  $\alpha < \alpha_{\text{low}}$  then ▷ High volatility: ShiftGear
   liveness-first phase
6:   return {phase: ShiftGear,  $t_1 \leftarrow$ 
      $t_{\text{res}}$ , bba_timeout  $\leftarrow$  volatile_timeout( $s$ )}
7: else if network_healing = True then ▷ Network
   recovery: atomic healing phase
8:   return
   {phase: AnchorRecovery,  $t_2, t_3$ , recovery_window}
9: else ▷ Hybrid transition: unified clamped sigmoid
   interpolation
10:   $t_1 \leftarrow$ 
    clamp( $\sigma(\alpha) \cdot t_{\text{sta}} + (1 - \sigma(\alpha)) \cdot t_{\text{res}}$ ,  $t_{\text{sta}}$ ,  $t_{\text{res}}$ )
11:  return {phase: Hybrid,  $t_1$ }
12: end if

```

MT-BFT Clamped Threshold Calibration vs. Binary Threshold Switching: Traditional BFT protocols rely on fixed quorum thresholds or empirically tuned discrete switching rules, leading to unavoidable performance tradeoffs and inconsistent safety guarantees in dynamic network environments. Although our design adopts continuous threshold adjustment to balance throughput and resilience, all dynamic quorum values are strictly bounded within the provably secure envelope $[t_{\text{sta}}, t_{\text{res}}]$ under the standard 1/3 Byzantine fault model and derived from formal clamped sigmoid interpolation. This unified theoretical constraint eliminates potential ambiguity between intuitive volatile-mode threshold settings and formal security invariants, ensuring adaptive policies are theoretically grounded rather than empirical engineering choices.

VTP-BBA vs. Traditional BBA Paradigms: The proposed VTP-BBA integrates topology pruning, versioned voting, and cross-phase state coordination to improve consensus convergence. It is worth noting that topology reachability detection and subgraph reconstruction introduce marginal auxiliary overhead, which is an inherent limitation of topology-adaptive designs. However, these operations are executed as periodic asynchronous background updates instead of being invoked in every consensus round. Since network topology and shard membership evolve far slower than high-frequency consensus iterations, topological reconstruction is triggered infrequently. The one-time computational cost can be fully amortized over successive consensus rounds, resulting in negligible per-round overhead. By eliminating invalid node participation and redundant voting iterations in advance, the consensus still sustains stable $O(n)$ linear communication complexity, achieving practical improvements in dynamic adaptability. All auxiliary topological overhead is amortized and does not influence the asymptotic linear complexity of core consensus.

Threat Model and Problem Scope Clarification: This design rigorously distinguishes between practical application scenarios and formal security boundaries. The design is motivated by real-world sharding challenges, including dynamic shard composition and transient network partitioning induced by node churn, which invalidate static BFT assumptions. Nevertheless, our framework strictly adheres to the standard Byzantine threat model ($n \geq 3f + 1$, at most $1/3$ adversarial nodes per shard) for all formal security proofs. All adaptive mechanisms enhance system resilience against transient network instability and temporary node unavailability while strictly preserving fundamental Byzantine fault tolerance bounds.

V. THEORETICAL ANALYSIS

This section presents formal security proofs and asymptotic complexity analysis for the core mechanisms of DPC-Sharding.

This work targets network volatility and node churn as the practical system scenario, while all formal security analysis strictly adheres to the standard snapshot Byzantine fault model. At any stable system snapshot, each shard satisfies the classic $n \geq 3f + 1$ constraint with at most f Byzantine nodes. Network dynamics only change online node sets and network connectivity, rather than breaking the $f < n/3$ adversarial boundary. The proposed multi-threshold adaptive strategy is precisely designed to maintain continuous consensus safety and liveness under time-varying honest node availability, ensuring full alignment between problem motivation, threat model, and formal proofs.

We theoretically validate the safety of the MT-BFT safety envelope, clamped sigmoid interpolation rule, adaptive consensus invariants, LSSM state transition, topology-aware convergence, cross-shard atomic execution, partition recovery convergence, cross-phase global consistency, and comprehensive asymptotic complexity. The whole system is divided into multiple independent non-overlapping shards, and all theorems are extended and adapted for sharding architecture.

A. Fundamental Quorum Intersection Property

Quorum intersection is the fundamental invariant to eliminate conflicting committed states in Byzantine consensus. In our sharding system, each node belongs to exactly one shard to avoid cross-shard identity multiplexing interference.

Lemma 5.1: Under $n \geq 3f + 1$, a quorum threshold t satisfies the Byzantine quorum intersection property if $t \geq n - f$.

Proof: We first prove sufficiency by contradiction. Assume two disjoint valid quorums Q_1, Q_2 exist, such that $|Q_1| \geq n - f$, $|Q_2| \geq n - f$, and $Q_1 \cap Q_2 = \emptyset$. The disjoint constraint yields $|Q_1| + |Q_2| \leq n$. Substituting the threshold condition gives $|Q_1| + |Q_2| \geq 2(n - f)$. Combining $n \geq 3f + 1$, we obtain $2(n - f) = 2n - 2f \geq n + f + 2 > n$. This contradicts $|Q_1| + |Q_2| \leq n$. Thus, any two quorums of size $t \geq n - f$ must intersect.

By Lemma 5.1, all thresholds within the MT-BFT closed safe interval $[t_{\text{sta}}, t_{\text{res}}]$ satisfy the quorum intersection property at the resilient bound.

Lemma 5.2: After volatility-aware topology pruning, the derived connected subgraph G^* still satisfies the fundamental Byzantine fault tolerance constraint $|G^*| \geq 3f + 1$.

Proof: The topology pruning operation only eliminates unreachable, timeout and permanently offline honest nodes, rather than removing any Byzantine nodes inside the shard. All adversarial nodes are fully retained in the constructed subgraph G^* . Let the original shard satisfy $n \geq 3f + 1$ with exactly f malicious nodes.

Denote the number of eliminated offline honest nodes as k , where $k \geq 0$. The node scale of the pruned subgraph is $|G^*| = n - k$. Since the protocol only removes completely isolated nodes that cannot participate in consensus, the system retains sufficient online honest nodes to maintain basic consensus availability. According to the valid shard definition in Definition 2.2, the pruning strategy will not excessively reduce online honest nodes. Thus, the remaining honest nodes still satisfy $|G^*| - f \geq 2f + 1$, which yields $|G^*| \geq 3f + 1$.

Therefore, the valid subgraph generated by VTP-BBA topology pruning still follows the standard $n \geq 3f + 1$ fault tolerance assumption, and all subsequent consensus operations inside G^* are compatible with the MT-BFT safety model.

B. Validity Analysis of MT-BFT Safety Envelope

The MT-BFT framework defines two rigorous boundary thresholds to constrain all adaptive consensus parameters, unifying synchronous consistency guarantees and asynchronous liveness guarantees within one formal model. All floating threshold outputs are rounded to valid integers to satisfy practical voting rules.

Theorem 5.1: Given $n \geq 3f + 1$, the steady-state threshold $t_{\text{sta}} = \lceil 2n/3 \rceil$ guarantees strong consensus consistency, and the resilient threshold $t_{\text{res}} = n - f$ guarantees consensus liveness under network partitions and honest node fluctuation.

Proof: For $t_{\text{sta}} = \lceil 2n/3 \rceil$: Let any two valid quorums be Q_a, Q_b with $|Q_a|, |Q_b| \geq 2n/3$. The intersection size satisfies $|Q_a \cap Q_b| = |Q_a| + |Q_b| - |Q_a \cup Q_b| \geq n/3$. Under $n \geq 3f + 1$, we have $n/3 > f$. The overlapping honest nodes dominate the intersection, so no conflicting commit can be forged by Byzantine nodes, which guarantees strict consistency.

For $t_{\text{res}} = n - f$: At most f nodes are adversarial. The honest node set contains at least $n - f$ online nodes, which can always form a valid quorum regardless of message delay or temporary network segmentation. This threshold preserves complete Byzantine resilience and serves as the security bound for adaptive adjustment.

C. Security Verification of Clamped Sigmoid Interpolation

To avoid abrupt binary switching and performance cliffs, DPC-Sharding adopts clamped sigmoid interpolation for continuous threshold adaptation. All dynamic thresholds are strictly bounded within the MT-BFT safe interval, and the continuous adaptation is fully formalized rather than heuristic engineering tuning.

Theorem 5.2: Let $\alpha \in [0, 1]$ denote network stability confidence and $\sigma(\alpha)$ be the standard sigmoid function. The adaptive quorum threshold is defined as:

$$t(\alpha) = \text{clamp}(\sigma(\alpha) \cdot t_{\text{sta}} + (1 - \sigma(\alpha)) \cdot t_{\text{res}}, t_{\text{sta}}, t_{\text{res}}).$$

All arbitrary threshold values generated by $t(\alpha)$ within $[t_{\text{sta}}, t_{\text{res}}]$ satisfy Byzantine safety invariants, including quorum intersection, consistency, and liveness. The function maintains non-decreasing smooth adaptation and exact boundary convergence under discrete network sampling.

Proof: The clamp function strictly restricts $t(\alpha) \in [t_{\text{sta}}, t_{\text{res}}]$. Every value in this closed interval satisfies the quorum intersection property, which constitutes the core safety invariant for Byzantine consensus. Therefore, all interpolated transition thresholds during network fluctuation are provably safe, with no risky under-bound quorum requirements.

The sigmoid function is strictly increasing, making the weighted combination monotonically varying with network stability. The clamp operation only fixes boundary values without introducing unsafe intervals, ensuring global non-decreasing smooth adaptation.

For discrete system iteration: network stability α is updated periodically, and the threshold converges precisely to t_{sta} under stable networks and t_{res} under volatile networks. The discrete adaptation complies with the continuous formal constraints, securing system safety across all dynamic transition phases.

D. Correctness of LSSM Network State Transition

The LSSM module performs network volatility quantification independently per shard, enabling shard-level adaptive adjustment to fit differentiated local network conditions instead of global uniform judgment. This work assumes that Byzantine nodes cannot tamper with multi-round topological sampling results, ensuring state transition reliability.

Theorem 5.3: Under $n \geq 3f + 1$, shard-level network state transitions determined by LSSM cannot be manipulated by an adversary controlling fewer than f nodes.

Proof: Each node generates binary volatility labels based on multi-round topological sampling. A state transition is triggered only when at least $n - f$ intra-shard nodes reach identical judgments. Since honest nodes dominate the $n - f$ threshold and adversaries control at most f nodes, malicious collusion cannot forge sufficient unified labels to manipulate switching behavior. The embedded hysteresis window further filters transient network jitter, guaranteeing stable and secure mode transition for dynamic shard topologies.

E. Bounded Convergence of VTP-BBA Consensus Protocol

In dynamic networks, topology pruning is adopted to eliminate permanently offline nodes while retaining temporarily unstable honest nodes, which preserves sufficient node redundancy to maintain the $n \geq 3f + 1$ fault-tolerance boundary. The pruning procedure includes periodic reachability detection, intra-shard node set synchronization, and connected subgraph reconstruction. All auxiliary topology maintenance overhead is fully incorporated into complexity analysis.

Theorem 5.4: Constrained by the MT-BFT safety envelope, the proposed VTP-BBA protocol with topology pruning and versioned iteration achieves deterministic bounded convergence and eliminates livelock under dynamic network conditions.

Proof: Topology pruning continuously expels permanently disconnected nodes and restricts consensus interactions within valid connected subgraphs, isolating invalid message interference from offline or malicious nodes. All voting and commit behaviors are strictly validated follow the t_{res} safety bound, ensuring each consensus iteration is supported by sufficient honest node quorums.

Specifically, the temporary half-majority threshold $\lceil |G^*|/2 \rceil$ adopted for intra-subgraph preliminary screening never participates in formal consensus commit decisions. It serves merely as a lightweight node eligibility filter to accelerate voting convergence under severe network partitioning. All final consensus confirmation and state settlement strictly follow the unified resilient safety bound $t_{\text{res}} = n - f$.

This two-stage paradigm decouples lightweight pre-filtering from safety-critical formal confirmation. The half-majority rule only screens candidate nodes in advance and never replaces the standard MT-BFT quorum. Therefore, the hybrid design preserves the quorum intersection property and introduces no additional Byzantine security risks.

The protocol adopts a round-based iteration structure with globally unified version vectors. All voting messages are tagged with monotonically increasing sequence identifiers to discard stale, conflicting, and out-of-order votes.

The protocol version number is non-decreasing and bounded by finite iteration rounds and global checkpoint thresholds. Each successful consensus iteration strictly increments the version number, and fewer than f Byzantine nodes cannot permanently reverse or halt version growth. Since the total number of reachable system states is finite, infinite cyclic state transitions (livelock) are impossible. Hence, VTP-BBA achieves deterministic bounded convergence under dynamic network volatility.

Corollary 5.1: Strict monotonic increment of version numbers is a necessary condition for intra-shard consensus progress, and version stagnation is sufficient to ensure global state agreement and eliminate divergence among honest nodes under Byzantine constraints.

Proof: Any committed state is certified by valid quorums complying with Lemma 5.1, ensuring non-empty intersection containing honest nodes. Conflicting versions cannot coexist in the system. When version numbers stop updating, all honest nodes stabilize at the latest verified checkpoint, forming a globally consistent state.

F. Atomicity of Cross-Shard Transactions

For cross-shard transactions, the system adopts a max-threshold weakest-link strategy with explicit upper-bound constraints to balance consistency and liveness, avoiding excessive performance degradation while provably guaranteeing transaction execution correctness.

Theorem 5.5: The versioned lock mechanism guarantees atomicity for cross-shard transactions under hierarchical MT-BFT threshold constraints.

Proof: For any cross-shard transaction involving multiple independent shards, the protocol adopts the maximum quorum threshold t_{max} among all participating shards for final commit

confirmation. Since each shard's adaptive threshold is constrained within the safe interval $[t_{\text{sta}}, t_{\text{res}}]$, the derived t_{max} always satisfies the quorum intersection invariant in Lemma 5.1, thus retaining complete Byzantine safety and preventing conflicting cross-shard commits.

The versioned lock mechanism attaches globally unique monotonic timestamps to all cross-shard transaction states, strictly prohibiting outdated state overwriting and ensuring causal execution ordering across shards. During network fluctuation or temporary partition, any inconsistent local shard state is rolled back uniformly to the latest cryptographically verified global checkpoint certified by valid honest quorums. The protocol enforces a strict all-or-nothing execution paradigm: either all involved shards complete consistent state commit, or the entire transaction is rolled back without partial state persistence. This mechanism eliminates partial commit anomalies and strictly satisfies the ACID atomicity requirement for cross-shard transaction execution.

The protocol guarantees all-or-nothing execution atomicity, while global serializability follows from the ordered state machine replication deployed within each shard.

Theorem 5.6: The intra-shard consensus threshold, cross-shard validation threshold and global checkpoint threshold adopted by DPC-Sharding are uniformly constrained under the MT-BFT safety envelope, and the hierarchical multi-threshold design maintains consistent Byzantine safety without internal security conflicts.

Proof: We analyze the security of three layers separately based on Lemma 5.1 and Theorem 5.2:

1) *Layer 1 Intra-shard Dynamic Consensus Threshold:* All adaptive thresholds generated by clamped sigmoid interpolation are confined within $[t_{\text{sta}}, t_{\text{res}}]$. As proven in Lemma 5.1, all values within this interval satisfy quorum intersection property, ensuring fundamental consensus safety for single-shard transaction ordering.

2) *Layer 2 Cross-shard Validation Threshold:* The cross-shard threshold follows the weakest-link principle $t_2 = \max(t_1^{(i)})$. When network conditions degrade to asynchronous partitions, the maximum value across shards converges to $n - f$ and the threshold $n - f$ still meets the quorum intersection requirement, avoiding inconsistent cross-shard commit states.

3) *Layer 3 Global Checkpoint Threshold:* The fixed checkpoint threshold is set as $t_3 = 2f + 1$. Combined with $n \geq 3f + 1$, we have $2f + 1 \leq n - f$. This fixed threshold falls inside the MT-BFT safe envelope and provides stable state anchoring without violating Byzantine security invariants.

In summary, all hierarchical thresholds follow the same unified MT-BFT safety constraints. The layer-wise threshold difference only corresponds to differentiated functional requirements rather than inconsistent security standards.

G. Partition Healing Convergence and Cross-Phase Global Consistency

To complete full-lifecycle correctness verification, we further prove convergence properties after network partition recovery and state consistency across multi-phase execution.

Theorem 5.7: The AnchorRecovery mechanism enables the system to converge to a unique global consistent state within finite rounds after network partitions recover, eliminating persistent state divergence and asynchronous livelock risks.

Proof: All transaction states carry monotonically increasing version identifiers and are anchored to cryptographically verified global checkpoints. Nodes reject outdated state messages to avoid stale data propagation. Divergent local states are uniformly rolled back to the latest valid checkpoint certified by honest quorums. Transaction merging follows deterministic dependency rules, and the total reachable state space is finite. No cyclic state transition can persist, so the system eventually converges to one unified global state.

Theorem 5.8: Consensus results generated in different operational phases can be safely synchronized, and the whole execution stack maintains global consistency under unified MT-BFT safety invariants.

Proof: All phases abide by the identical safe threshold interval and interpolation rules. Consensus outputs are attached with verifiable signatures and version tags, resisting tampering by Byzantine nodes. State handover strictly follows quorum intersection and version monotonicity rules. No invalid state can cross phase boundaries, thus end-to-end global consistency is preserved throughout system operation.

H. Analysis of Asymptotic Complexity

The system overhead is decomposed into two categories:

1) *Core consensus overhead:* Intra-shard ordering, cross-shard validation, and global checkpointing, which run per consensus round. 2) *Topology-aided auxiliary overhead:* Asynchronous background operations including topology pruning, reachability detection, connected subgraph construction, and intra-shard node set exchange. These operations are triggered intermittently rather than in every consensus round.

1) *Communication Complexity:* Intra-shard consensus adopts a fully peer-to-peer voting model with linear message propagation, yielding a per-round communication complexity of $O(n)$. Cross-shard validation leverages compressed Merkle proofs and achieves constant $O(1)$ cross-shard interaction complexity.

We formally analyze the complexity of each topology-aided auxiliary operation:

- *Topology pruning:* Local neighbor state scanning within each shard, resulting in $O(n)$ communication cost per execution.
- *Reachability detection:* Bounded intra-shard connectivity probing, incurring $O(n)$ communication cost per execution.
- *Subgraph G^* construction:* Local connectivity aggregation without global broadcasting, giving $O(n)$ communication cost per execution.
- *Node set exchange:* Restricted intra-shard view synchronization, introducing $O(n)$ communication cost per execution.

Since all auxiliary operations run asynchronously and are triggered only upon detectable network volatility, their overhead can be formally amortized over multiple consensus rounds.

Lemma 5.3: The four typical asynchronous topology operations, including topology pruning, reachability detection, subgraph G^* construction and intra-shard node set exchange, each incur $O(n)$ communication cost per execution. Let T denote the average interval (in consensus rounds) between two consecutive full topology updates. If $T = \Omega(n)$, the amortized per-round communication overhead of all auxiliary operations is $O(1)$.

Proof: Each individual topology operation costs $O(n)$ when executed. A complete topology update consists of all four operations, leading to an overall $O(n)$ cost per update. When amortized over T consecutive consensus rounds, the average overhead per round is $O(n)/T$. Under $T = \Omega(n)$, we obtain the amortized overhead $O(1)$ per round.

Theorem 5.9: Under typical network volatility (the targeted scenario of this work), the total amortized per-round communication complexity of DPC-Sharding is $O(n)$.

Proof: Core consensus contributes $O(n)$ communication cost per round. By Lemma 5.3, the amortized auxiliary overhead is $O(1)$. Summing these yields $O(n) + O(1) = O(n)$.

2) *Computation Complexity:* Local operations including threshold interpolation, version validation, and signature verification are constant-time $O(1)$. Multi-signature aggregation, hash computation, Merkle proof verification, and subgraph construction induce linear $O(n)$ computation cost. All cross-shard state synchronization and rollback operations are bounded by $O(n)$ computation complexity. Thus, the overall per-round computation complexity is $O(n)$.

3) *Complexity Under Dynamic Network Scenarios:* We analyze system complexity across three practical network regimes, based on the topology update interval T defined in Lemma 5.3. Recall that each full topology update incurs $O(n)$ communication cost, and the amortized overhead per consensus round is calculated as $O(n)/T$.

1) *Steady network state:* Topology adjustments are triggered very occasionally, leading to a large update interval satisfying $T \gg n$. In this case, $O(n)/T = O(1)$, so all auxiliary overhead is fully amortized. The overall communication and computation complexity remains bounded by $O(n)$.

2) *Slight-to-moderate fluctuation:* Under low-to-moderate network dynamics, nodes join or leave intermittently and transient partitions may occur. Topology updates run periodically, and the interval still satisfies $T = \Omega(n)$. The amortization mechanism continues to hold, and the overall complexity is still $O(n)$, with only a minor constant overhead introduced by periodic topology maintenance.

3) *Severe partition or high-frequency churn:* When persistent network partitions and intensive node churn dominate the network, the system needs to refresh topology information in nearly every consensus round. This results in $T = O(1)$, which breaks the amortization condition. The per-round communication cost of topology maintenance rises to $O(n)$. Combined with the $O(n)$ cost of core consensus, the total system complexity exceeds the linear bound. Note that linear complexity cannot

be guaranteed under these extreme regimes beyond our target scenarios.

4) *Latency Performance Bound:* Under stable networks, the adaptive threshold converges to $t_{\text{sta}} = \lceil 2n/3 \rceil$, achieving low-latency consensus with high throughput. Under volatile network conditions, the threshold shifts smoothly toward $t_{\text{res}} = n - f$ to preserve liveness, which moderately increases consensus latency. The LSSM module dynamically adjusts the latency tradeoff without violating safety invariants.

VI. IMPLEMENTATION AND EVALUATION

Overview: We implement DPC-Sharding and comparative approaches in Go (v1.20) and python. Critical latency calculations (e.g., volatility score ω_s and stability confidence α) are optimized using AVX2 instructions. Replicas joining a shard start participating in consensus (voting on new transactions) immediately after their membership request is approved; full state synchronization (via Merkle differences) runs in parallel, with only post-request execution deferred until transfer completes.

We conduct DPC-Sharding evaluation with 200 containerized nodes deployed on 10 high-performance physical servers (x86-64 cloud servers: 8-core Intel Xeon 2.3 GHz, 32 GB RAM each), interconnected via a 10 Gbps switch, with Linux tc injecting 10–500 ms log-normal latency and 0–20% bursty packet loss, and iptables inducing 2–5-subnet partitions; all protocols use gRPC v1.54, BLS signatures (BLS12-381 for CRS), SHA-256 hashing.

We use workloads including 512B IoT transactions, 1 KB Ethereum-like transactions, 2–5-shard cross-shard distribution. For DPC-Sharding, we tune LSSM parameters ($\omega_{\text{th}} = 0.15$, 3-epoch stability window), Multi-Threshold BFT ($t_1 = \lceil 2n/3 \rceil$, $t'_1 = n - f$, *sigmoid* steepness=10), and execution stack configuration (1000-tx FlashPath batch, 100 ms ShiftGear latency, 10/100-block AnchorRecovery checkpoint). Comparative Schemes follow original configurations. Basic metrics include throughput, consensus latency, shard reconfiguration delay, transaction success rate, attack violation ratio, reachability detection latency and dynamic subgraph construction cost. Each experiment lasts 10 minutes with 30 repeated runs, and final results are derived under the 95% confidence interval.

Comparative schemes follow original configurations as closely as possible under our experimental platform. Slight parameter adjustments are made to ensure compatibility with our deployment environment and workload, and all protocols are evaluated under identical network and hardware conditions. Due to inherent differences in hardware specifications, network latency settings and node scale, the absolute performance values of baseline schemes are slightly different from those reported in their original papers, while the overall variation trends and relative performance remains consistent.

The evaluation focuses on the following research questions regarding internal property verification and external performance comparison:

- *RQ1 (Framework Integrity & Adaptive Stability):* Can DPC-Sharding realize smooth adaptive operation without

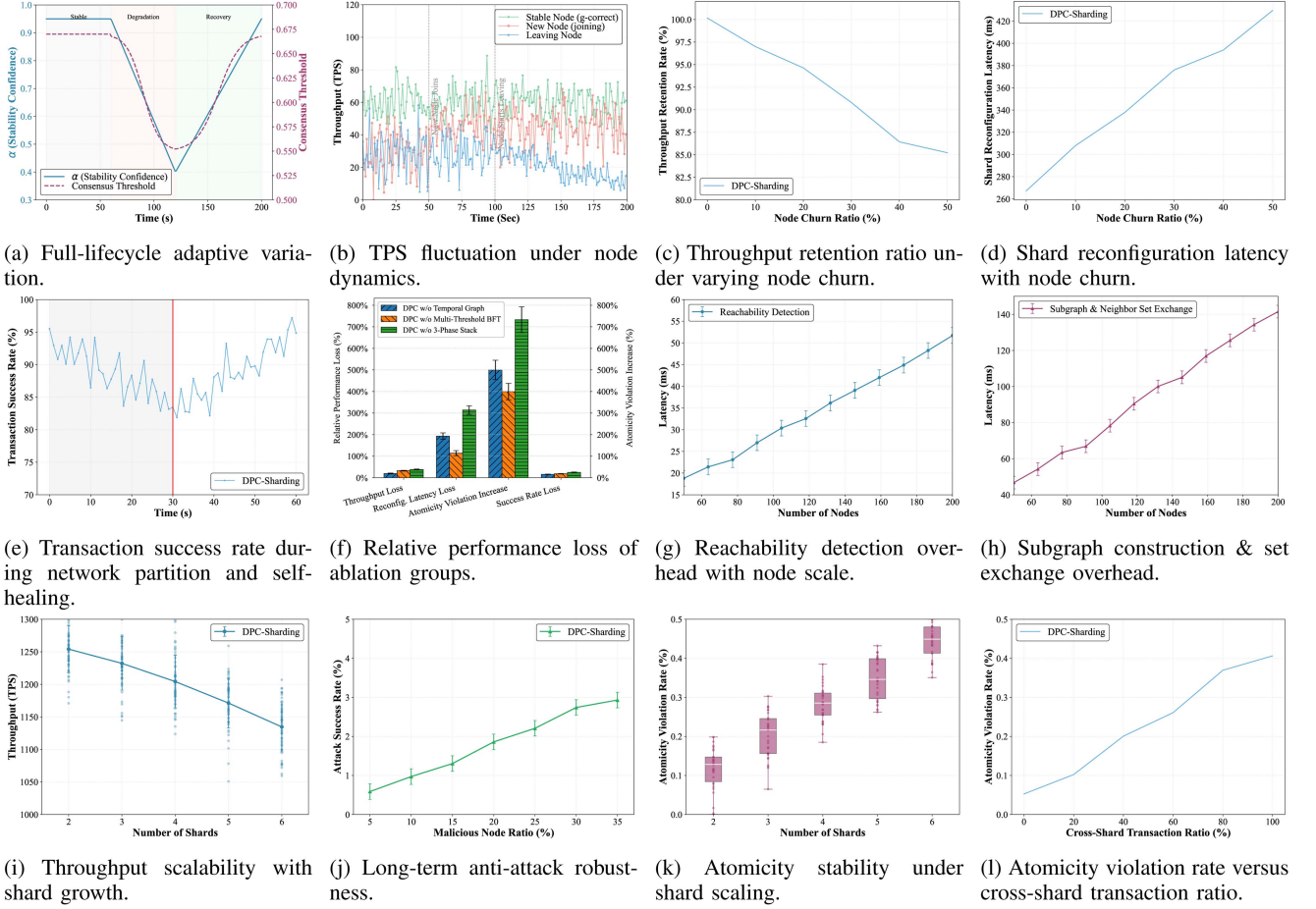


Fig. 4. DPC-Sharding adaptability, module synergy and security evaluation.

performance drops during normal operation and node churn, and guarantee the integrity and internal consistency of the overall sharding framework?

- *RQ2 (Module Synergy & Overhead Controllability)*: How do the three core modules (temporal graph sensing, MT-BFT threshold control, and three-phase execution stack) work individually and collaboratively? In addition, is the overhead brought by topology pruning lightweight and controllable? Does the system retain linear communication complexity after incorporating all auxiliary costs?
- *RQ3 (Security & Atomicity)*: Can DPC-Sharding sustain strong Byzantine resistance against adversarial nodes and maintain a low cross-shard atomicity violation rate as the number of shards increases?
- *RQ4 (Performance Comparison with Baseline Sharding Schemes)*: Compared with typical static sharding protocols (e.g., OmniLedger) and adaptive ones (e.g., DynaShard), what gains in throughput and latency can DPC-Sharding achieve under stable network conditions?

A. Verification of DPC-Sharding

We validate framework adaptability, module collaboration effect, overhead controllability and inherent security performance.

1) Framework Integrity & Adaptive Stability (RQ1): This subsection validates the **holistic end-to-end framework capability** covering the full pipeline: network sensing, adaptive threshold adjustment, three-phase execution switching, and partition self-recovery. System stability degrades gently with network deterioration and recovers steadily after network restoration. The consensus threshold adjusts smoothly without abrupt jumps, and throughput remains stable during node churn and network state transitions, verifying the non-cliff adaptability of the integrated framework.

Fig. 4(a) plots the continuous variation of stability confidence α and the adaptive consensus threshold over the full network lifecycle. The network gradually evolves from a stable state to a volatile phase and recovers smoothly, while the consensus threshold is adjusted in a sigmoid-bounded manner without abrupt jumps, reflecting the network sensing and threshold adaptation stages. Note that the displayed consensus threshold values are normalized proportions, and the practical node-count voting threshold strictly falls within the MT-BFT safe interval.

Fig. 4(b) further characterizes system throughput dynamics under node joining and leaving events. The throughput remains stable without obvious performance cliffs, demonstrating that the latency-sensitive state machine and threshold calibration enable smooth, oscillation-free adaptive scheduling during node dynamic changes.

TABLE I
ABLATION RESULTS OF CORE FUNCTIONAL MODULES

Index	Full DPC	w/o TG	w/o MT-BFT	w/o 3-Phase Stack
Throughput (TPS)	1220 $\pm$ 50	980 $\pm$ 40	820 $\pm$ 30	750 $\pm$ 35
Reconfig Delay (ms)	290 $\pm$ 10	850 $\pm$ 25	620 $\pm$ 20	1200 $\pm$ 40
Atomic Violation (%)	0.3 $\pm$ 0.1	1.8 $\pm$ 0.2	1.2 $\pm$ 0.2	2.5 $\pm$ 0.3
Success Rate (%)	97.2 $\pm$ 1.5	92.8 $\pm$ 2.0	90.5 $\pm$ 2.2	72.4 $\pm$ 3.5

Fig. 4(c) quantifies throughput retention under node churn. Benefiting from LSSM load perception and topology adaptive reorganization, DPC-Sharding maintains high throughput retention under normal dynamics, consistent with the framework's adaptive capability.

Fig. 4(d) shows that, relying on the lightweight ShiftGear adjustment mechanism and VTP-BBA optimization, shard reconfiguration latency grows gently under normal dynamics, confirming lightweight and controllable framework-level reconfiguration.

Fig. 4(e) depicts transaction success rate during partition and self-healing. Enabled by volatility-aware topology pruning and cross-shard version voting recovery, DPC-Sharding can still maintain a robust self-healing probability when the network partition scope expands, completing the final stage of the partition detection and recovery workflow.

2) *Module Synergy & Overhead Controllability (RQ2)*: This subsection conducts ablation experiments and topological overhead measurements, investigating the independent contribution of each core module and the lightweight property of topological maintenance.

Fig. 4(f) and Table I quantify the performance degradation resulting from disabling individual modules, thereby verifying the unique contribution of temporal graph sensing, MT-BFT threshold control, and the three-phase execution stack.

Fig. 4(g) and Fig. 4(h) show that the latency of reachability detection and subgraph construction increases gently with the number of nodes. However, since these topology maintenance tasks run as asynchronous background jobs, their overhead is effectively amortized across multiple consensus rounds. In typical network conditions, they do not become the main performance bottleneck.

Fig. 4(i) further illustrates that system throughput declines smoothly with the growth of shard number, preserving stable scalability and linear communication complexity under expanding network size.

These core adaptive components are essential for the system to adapt to routine network volatility, which is the targeted scenario of this work. Although the embedded standalone topology maintenance tasks introduce marginal extra overhead, such overhead can be well amortized across consensus rounds under typical network dynamics, maintaining stable linear amortized communication complexity. For extreme network conditions, including frequent shard reconfiguration and persistent network partitions, unamortized auxiliary overhead will accumulate and degrade system performance, whereas core Byzantine security properties are always preserved.

3) *Security & Atomicity (RQ3)*: This subsection evaluates inherent security and atomicity, independent of adaptive performance. All conclusions are constrained within the fault bound $f < n/3$.

As illustrated in Fig. 4(j), the attack success rate remains low even when malicious nodes reach 33%, verifying Byzantine fault tolerance within theoretical bounds.

Fig. 4(k) shows atomicity violation is controlled below 0.5% and rises moderately with shard scale. Fig. 4(l) shows that the atomicity violation rate rises marginally and remains within a low, stable interval with increasing cross-shard traffic. These results confirm that version locking and causal merging effectively guarantee cross-shard atomicity within tested scales.

B. Performance Comparison

We benchmark DPC-Sharding against static OmniLedger and adaptive DynaShard to address RQ4. All performance claims are limited to our experimental environment.

Fig. 5(a) compares the consensus latency and state-transfer latency. Benefiting from volatility-aware topology pruning and lightweight VTP-BBA design, DPC-Sharding simplifies intra-shard consensus decision and cross-shard state synchronization, thus reducing network coordination and transmission overhead compared with OmniLedger.

Fig. 5(b) illustrates throughput variation under growing transaction load. Constrained by fixed threshold and topology-agnostic voting, DynaShard suffers throughput degradation as system load rises. Equipped with adaptive threshold calibration and three-phase execution stack, DPC-Sharding suppresses performance attenuation and maintains stable and sustainable throughput output under heavy load.

Fig. 5(c) further verifies the peak throughput of three protocols in stable scenarios. As a permissionless large-scale sharding paradigm, OmniLedger possesses the highest theoretical throughput but at the cost of excessive latency and communication overhead. DPC-Sharding targets lightweight edge and consortium blockchain scenarios, achieving a moderate yet competitive peak throughput that outperforms DynaShard while striking a trade-off among throughput, latency and scalability.

Fig. 5(d) compares shard resource utilization before and after dynamic reconfiguration. While DynaShard relies primarily on workload indicators to trigger shard splitting and merging, DPC-Sharding complements load monitoring with LSSM awareness and ShiftGear adaptive tuning. It thus achieves more delicate load distribution and reduces the deviation from ideal utilization.

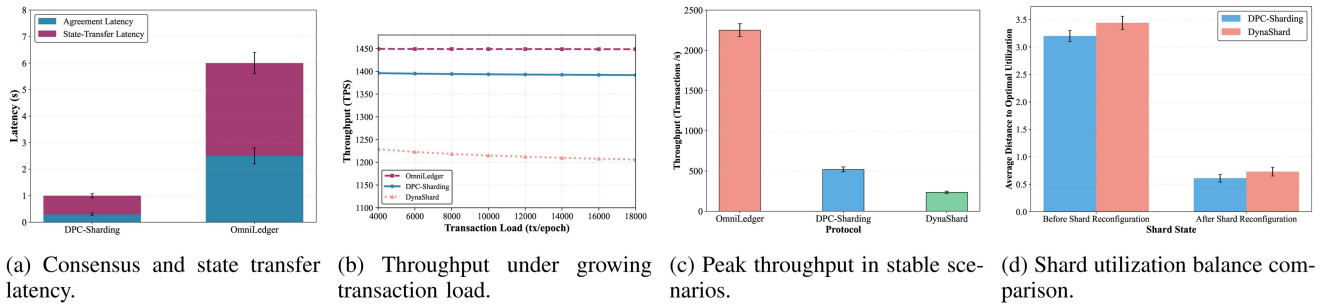


Fig. 5. Performance Comparison with baseline sharding solutions

VII. CONCLUSION

This paper proposes DPC-Sharding, an adaptive Byzantine-resilient sharding framework for time-varying partially connected blockchain networks. Built upon formal MT-BFT safety invariants, this framework integrates temporal graph awareness, latency-sensitive state machines and clamped multi-threshold BFT equipped with the VTP-BBA protocol. Different from existing sharding schemes that adopt binary mode switching and empirical thresholds to separately optimize throughput or network resilience, DPC-Sharding supports smooth continuous consensus adaptation to avoid dramatic performance degradation under network dynamics. Although it brings marginal overhead for topology pruning and subgraph reconstruction, such background cost can be fully amortized during successive consensus rounds, and the core consensus still maintains linear communication complexity. Both theoretical analysis and experimental results demonstrate that DPC-Sharding achieves better tradeoffs among throughput, latency and partition resilience, and stably guarantees cross-shard transaction atomicity in dynamic network environments. In future work, we will optimize lightweight temporal graph updating strategies, strengthen adversarial attack defense capabilities, conduct practical testbed verification, and extend this framework to more edge computing and IoT scenarios.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their constructive comments and suggestions.

REFERENCES

- [1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [2] A. D. Kshemkalyani and M. Singhal, "Distributed system model," in *Distributed Computing: Principles, Algorithms, and Systems*. Cambridge, U.K.: Cambridge Univ. Press, 2012, pp. 10–14.
- [3] C. D. T. Barros, M. R. F. Mendonça, A. B. Vieira, and A. Ziviani, "A survey on embedding dynamic graphs," *ACM Comput. Surv.*, vol. 55, no. 1, pp. 1–37, Nov. 2023.
- [4] R. Adhikari, C. Busch, and D. R. Kowalski, "Stable blockchain sharding under adversarial transaction generation," in *Proc. ACM Symp. Parallelism Algorithms Archit.*, New York, NY, USA, 2024, pp. 451–461.
- [5] B. Yin, Q. Xie, R. Xu, and K. Gu, "NSshard: Low-cross-shard sharding via account partitioning for blockchain-based IoT," *IEEE Internet Things J.*, vol. 12, no. 7, pp. 9019–9030, Apr. 2025.
- [6] M. Song et al., "AERO: Enhancing sharding blockchain via deep reinforcement learning for account migration," in *Proc. Web Conf.*, New York, NY, USA, 2025, pp. 706–716.
- [7] Y. Liu et al., "Building blocks of sharding blockchain systems: Concepts, approaches, and open problems," *Comput. Sci. Rev.*, vol. 46, Nov. 2021, Art. no. 100513.
- [8] D. Abadi, "Consistency tradeoffs in modern distributed database system design: Cap is only part of the story," *Computer*, vol. 45, no. 2, pp. 37–42, 2012.
- [9] S. Duan and H. Zhang, "Foundations of dynamic BFT," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 1317–1334.
- [10] J. Neu, S. Sridhar, L. Yang, and D. Tse, "Optimal flexible consensus and its application to ethereum," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2024, pp. 3885–3903.
- [11] A. Liu et al., "DynaShard: Secure and adaptive blockchain sharding protocol with hybrid consensus and dynamic shard management," *IEEE Internet Things J.*, vol. 12, no. 5, pp. 5462–5475, Mar. 2025.
- [12] Y. Lu, Z. Lu, and Q. Tang, "Bolt-dumbo transformer: Asynchronous consensus as fast as the pipelined BFT," in *Proc. ACM Conf. Comput. Commun. Secur.*, Los Angeles, CA, USA, 2022, pp. 1–15.
- [13] X. Dai, B. Zhang, H. Jin, and L. Ren, "ParBFT: Faster asynchronous BFT consensus with a parallel optimistic path," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2023, pp. 504–518.
- [14] I. Abraham, D. Malkhi, K. Nayak, L. Ren, and M. Yin, "Sync Hot-Stuff: Simple and practical synchronous state machine replication," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2020, pp. 106–118.
- [15] M. Zamani, M. Movahedi, and M. Raykova, "RapidChain: Scaling blockchain via full sharding," in *Proc. ACM Conf. Comput. Commun. Secur.*, New York, NY, USA, 2018, pp. 931–948.
- [16] A. Miller et al., "The honey badger of BFT protocols," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2016, pp. 31–42.
- [17] B. Guo et al., "Dumbo: Faster asynchronous BFT protocols," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2020, pp. 803–818.
- [18] A. Momose and L. Ren, "Multi-threshold byzantine fault tolerance," in *Proc. ACM Conf. Comput. Commun. Secur.*, New York, NY, USA, 2021, pp. 1686–1699.
- [19] E. Kokoris-Kogias et al., "OmniLedger: A secure, scale-out, decentralized ledger via sharding," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2018, pp. 583–598.
- [20] L. Jia, Y. Liu, K. Wang, and Y. Sun, "Estuary: A low cross-shard blockchain sharding protocol based on state splitting," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 3, pp. 405–420, Mar. 2024.
- [21] Q. Zhang, S. Wang, D. Zhang, J. Wang, and J. Sun, "FortunChain: EC-VRF-based scalable blockchain system for realizing state sharding," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4340–4353, Dec. 2023.
- [22] H. Huang et al., "BrokerChain: A cross-shard blockchain protocol for account/balance-based state sharding," in *Proc. IEEE INFOCOM*, 2022, pp. 1968–1977.
- [23] H. Huang, Q. Zhao, and X. Ran, "Secure sharding scheme of blockchain based on reputation," in *Proc. IEEE Int. Conf. Data Sci. Comput. Appl.*, Dalian, China, 2022, pp. 1324–1327.
- [24] X. Huang et al., "ContribChain: A stress-balanced blockchain sharding protocol with node contribution awareness," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2025, pp. 1–10.
- [25] P. Song, Y. Liu, J. Chen, J. He, and Y. Wang, "Komorebi: A DAG-based asynchronous BFT consensus via sharding," in *Proc. IEEE Symp. Comput. Commun.*, Gammarth, Tunisia, 2023, pp. 1221–1227.

- [26] X. Wang, X. Sui, S. Duan, and H. Zhang, "Practical signature-free multi-valued validated Byzantine agreement and asynchronous common subset in constant time," *IEEE Trans. Comput.*, vol. 74, no. 12, pp. 4138–4152, Dec. 2025.
- [27] T. Cai, W. Chen, J. Zhang, and Z. Zheng, "SmartChain: A dynamic and self-adaptive sharding framework for IoT blockchain," *IEEE Trans. Serv. Comput.*, vol. 17, no. 2, pp. 674–688, Mar./Apr. 2024.
- [28] G. Leduc, S. Kubler, and J.-P. Georges, "Sabine: Self-adaptive blockchain consensus," in *Proc. 9th Int. Conf. Future Internet Things Cloud*, 2022, pp. 234–240.
- [29] C. Berger, L. Rodrigues, H. P. Reiser, V. Cogo, and A. Bessani, "Chasing lightspeed consensus: Fast wide-area Byzantine replication with mercury," in *Proc. 25th Int. Middleware Conf.*, 2024, pp. 158–171.
- [30] A. Casteigts, P. Flocchini, W. Quattrociocchi, and N. Santoro, "Time-varying graphs and dynamic networks," *Int. J. Parallel, Emergent Distrib. Syst.*, vol. 27, no. 5, pp. 387–408, Oct. 2012.
- [31] P. Holme, "Modern temporal network theory: A colloquium," *Eur. Phys. J. B*, vol. 88, no. 9, pp. 1–30, Sep. 2015.
- [32] Y. Yang, H. Wan, and J. Zhou, "DyDoGNN: A dynamic graph neural network with dower persistent homology for link prediction," in *Proc. Int. Conf. Artif. Intell., Internet Digit. Economy*, 2025, pp. 250–254.
- [33] X. Liu, H. Pu, Z. Chen, Z. Zhang, Y. Lan, and C. Shen, "EvolGCN: A co-evolutionary graph convolutional network model for dynamically spatio-temporal anomaly event inference," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 5, pp. 4503–4515, Sep./Oct. 2025.
- [34] J. Wang, J. Wang, X. Zhang, H. Du, and C. Jiang, "A time-varying graph-based dynamic blockchain sharding scheme for large-scale drone networks," *IEEE Internet Things J.*, vol. 13, no. 5, pp. 7960–7971, Mar. 2026.
- [35] B. Yin, P. Zhang, and T. Chen, "Efficient state sharding in blockchain via density-based clustering," *ACM Trans. Web*, vol. 19, no. 1, pp. 1–28, Feb. 2025.
- [36] A. Kumar et al., "Adaptive practical Byzantine fault tolerance with dynamic checkpointing," in *Proc. Int. Conf. Comput. Model., Simul. Optim.*, Singapore, 2025, pp. 202–206.
- [37] J. Huang et al., "BlockSense: Towards trustworthy mobile crowdsensing via proof-of-data blockchain," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1016–1032, Feb. 2024.
- [38] J. Huang, L. Kong, J. Wang, G. Chen, J. Gao, and G. Huang, "Secure data sharing over vehicular networks based on multi-sharding blockchain," *ACM Trans. Sens. Netw.*, vol. 20, no. 2, pp. 1–23, Mar. 2024.
- [39] H. Fang et al., "A novel network delay prediction model with mixed multi-layer perceptron architecture for edge computing," in *Proc. Int. Conf. Netw. Service Manage.*, Thessaloniki, Greece, 2022, pp. 191–197.
- [40] C. Avasalcai, C. Tsigkanos, and S. Dustdar, "Decentralized resource auctioning for latency-sensitive edge computing," in *Proc. IEEE Int. Conf. Edge Comput.*, Milan, Italy, 2019, pp. 72–76.
- [41] A. S. M. S. Hosen et al., "Blockchain-based transaction validation protocol for a secure distributed IoT network," *IEEE Access*, vol. 8, pp. 117266–117277, 2020.
- [42] H. Pan, C. Qu, H. W. Zhang, S. Wang, and J. Li, "HyperChain: A dynamic state sharding protocol supporting smart contracts to achieve low cross-shard and scalability," in *Proc. IEEE Int. Conf. Trust Secur. Privacy Comput. Commun.*, London, U.K., 2023, pp. 241–248.
- [43] R. Chen, H. Luo, G. Sun, H. Yu, D. Niyato, and S. Dustdar, "DRDST: Low-latency DAG consensus through robust dynamic sharding and tree-broadcasting for IoV," *IEEE Trans. Mobile Comput.*, vol. 25, no. 1, pp. 1236–1253, Jan. 2026.
- [44] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Gold Coast, QLD, Australia, 2001, pp. 514–532.
- [45] Y. Fan et al., "DRAS: Deep reinforcement learning for cluster scheduling in high performance computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4903–4917, Dec. 2022.
- [46] H. Howard, A. Charapko, and R. Mortier, "Fast flexible paxos: Relaxing quorum intersection for fast paxos," in *Proc. Int. Conf. Distrib. Comput. Netw.*, New York, NY, USA, 2021, pp. 186–190.
- [47] L. Alvisi, D. Malkhi, E. Pierce, and M. K. Reiter, "Fault detection for Byzantine quorum systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 12, no. 9, pp. 996–1007, Sep. 2001.
- [48] M. Arakawa and K. Shudo, "Block interval adjustment based on block propagation time in a blockchain," in *Proc. IEEE Int. Conf. Blockchain*, Espoo, Finland, 2022, pp. 202–207.
- [49] V. N. Moothedath, J. P. Champati, and J. Gross, "Energy efficient sampling policies for edge computing feedback systems," *IEEE Trans. Mobile Comput.*, vol. 22, no. 8, pp. 4634–4647, Aug. 2023.
- [50] B. He et al., "Region partitioning-based scalable real-time network verification via native distributed architecture," *IEEE Trans. Netw.*, vol. 34, no. 6, pp. 4621–4636, 2026.



Zhujun Zhang received the PhD degree from the School of Cyber Security, Chinese Academy of Sciences, Beijing, China, in 2021. She is currently an engineer with the Institute of Information Engineering, Chinese Academy of Sciences. Her research interests include blockchain consensus security, blockchain technology applications, and information security.



Wei Fan received the PhD degree from the School of Cyber Security, Chinese Academy of Sciences, Beijing, China, in 2018. He is currently a senior engineer with the Institute of Information Engineering, Chinese Academy of Sciences. His research focuses on blockchain security and cybersecurity technologies.



Zhiquan Liu (Senior Member, IEEE) received the BS degree from the School of Science, Xidian University, Xi'an, China, in 2012, and the PhD degree from the School of Computer Science and Technology, Xidian University, in 2017. He is currently a full professor, doctoral supervisor, and deputy dean with the College of Cyber Security, Jinan University, Guangzhou, China. His research focuses on security, trust, privacy, and intelligence in vehicular networks, and UAV networks. He currently serves as the area editor or associate editor of multiple SCI-index journals, such as *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Industrial Informatics*, *IEEE Internet of Things Journal*, *Information Fusion*, *Computer Networks*, *Journal of Systems Architecture*, and *Ad Hoc Networks*. His homepage is <https://www.zqliu.com>.